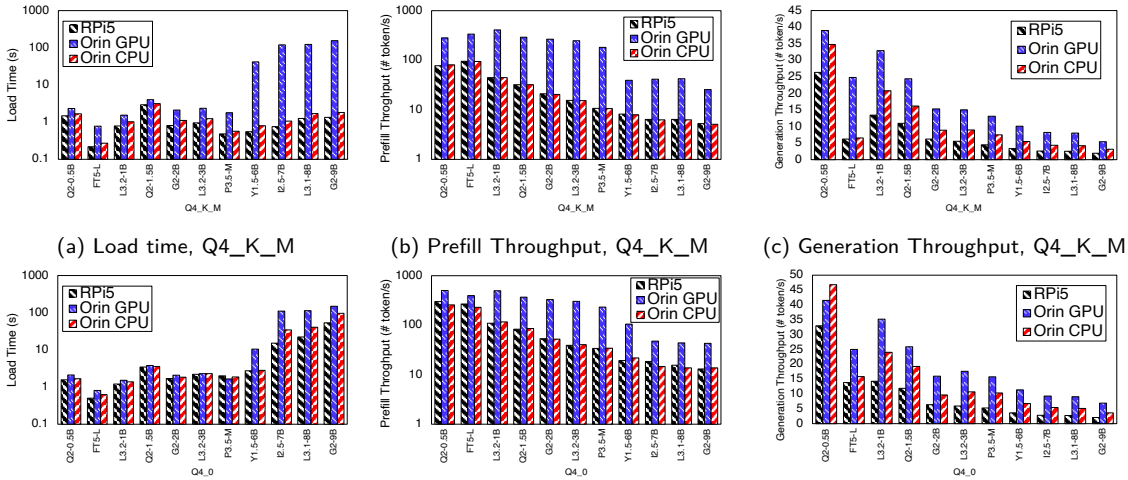



(a) Load time, Q4_K_M (b) Prefill Throughput, Q4_K_M (c) Generation Throughput, Q4_K_M

(d) Load Time, Q4_0 (e) Prefill Throughput, Q4_0 (f) Generation Throughput, Q4_0

Fig. 8. Load time and throughput for all models, quantization schemes, and all devices

in our experiments for the generation phase as is observable from Figure 6, which explains the degrading performance with larger thread counts beyond a certain point during the generation phase.

For GPU inference, the *15W* power mode improves both prefill by factors of $\sim 1.9 - 2.5$ and generation throughput by factors of $\sim 1.3 - 2.3$ compared to *7W* mode. Using maximum GPU frequencies further enhances prefill throughput by $\sim 1 - 58\%$, and generation throughput by $\sim 2 - 12\%$ improvements. Generally, a trend can be identified that the *MAX*-mode yields more pronounced performance improvements for smaller models than for larger models. For two configurations — Llama 3.1 8B with *Q4_0* quantization and Gemma 2 9B with *Q4_K_M* quantization — even small performance drops in prefill throughput were observable.

Looking at Figure 8, it is evident that load time is higher for GPU inference with *Q4_K_M* quantization. However, with *Q4_0* quantization, GPU load time is shorter for most models up to Phi 3.5 Mini, as CPU loading times increase by factors ranging from $\sim 1.05\times$ for Qwen 2 0.5B to $\sim 4\times$ for Phi 3.5 Mini. For larger models, such as Yi 1.5 6B and beyond, load times increase dramatically, with factors between $\sim 17\times$ and $\sim 53\times$ compared to *Q4_K_M* quantization. This occurs because the high memory footprint forces models to be evicted from memory during runs, requiring them to be reloaded from storage for each inference, significantly degrading performance. Additionally, a higher number of page faults was observed for these larger models, further increasing load times due to the slow nature of storage accesses. For *Q4_K_M* quantization, this issue was only noticeable in GPU inference. However, for *Q4_0* quantization, it also affected CPU inference, which can be explained by the increased memory footprint, as discussed in Subsection 5.1.

Figure 8 reveals that GPU inference consistently delivers the highest prefill throughput across all models and quantization schemes. In contrast, CPU inference shows comparable performance between the RPi 5 and Orin, except for Qwen 2 0.5B and Flan T5 Large under *Q4_0* quantization, where the RPi 5 achieves noticeably better results. A general trend of decreasing prefill throughput with increasing model size can be observed. However, Llama 3.2 1B stands out as an exception, significantly exceeding expectations in GPU

inference despite its relatively small size. Interestingly, this advantage was not present in CPU inference, contradicting Meta's claim that Llama 3.2 models are optimized for Arm processors [4]. Additionally, the largest models suffer a sharp drop in prefill performance, a consequence of their high memory footprint, as discussed earlier.

GPU inference achieves the highest generation throughput across all models, except for Qwen 2 0.5B under *Q4_0* quantization. However, the performance gap between CPU and GPU inference is much smaller than in the prefill phase. This can be attributed to the memory-bound nature of the generation phase, where increased computational power has less impact on performance improvements. For CPU inference, the Orin significantly outperforms the RPi 5, with throughput gains of $32-67\%$ for *Q4_K_M* quantization and $\sim 43-94\%$ for *Q4_0* quantization, depending on the model. However, for Flan T5 Large, this advantage is much less pronounced, with only $\sim 5\%$ and $\sim 14\%$ increases, respectively. This behavior can be explained by the Orin's larger cache size, which reduces cache misses, as seen in Figure 6. Due to the memory boundness of the generation phase a lower number of cache misses and, hence, faster memory access times have a significant impact on performance. Based on this result we believe, that there is room for optimizing the memory hierarchy and optimizing memory accesses for LM inference on future-generation edge devices to achieve faster token generation speeds.

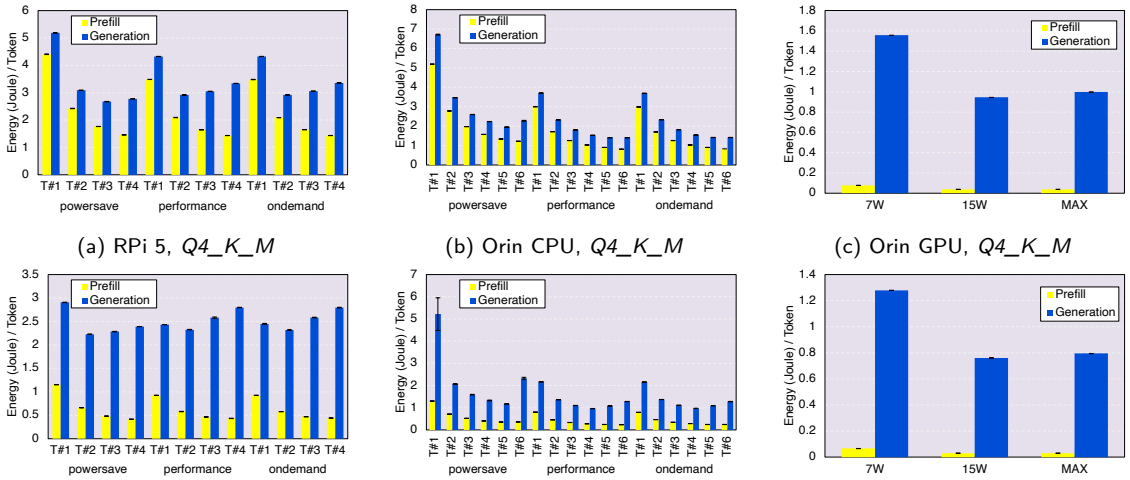
When comparing model performance, the general trend of decreasing throughput with increasing model size remains evident. Notable exceptions include Flan T5 Large and Gemma 2 models, which underperform relative to their size expectations. Unlike in the prefill phase, no major performance drops were observed for the largest models during generation.

**Main Takeaways**

- GPU inference outperforms CPU inference in terms of token throughput at the cost of higher load times
- *Q4_0* quantization substantially improves token throughput for all devices but induces higher load times for CPU inference than *Q4_K_M* quantization
- Token throughput follows approximately a hyperbolic relationship with model size

### 5.3 Energy Consumption and Efficiency Trade-Offs

In this subsection, we analyze the energy efficiency of LM inference across different configurations. Energy consumption is a critical factor in edge systems, particularly for battery-powered devices, where optimizing power usage directly impacts device longevity and real-world feasibility. Figure 9 illustrates the energy consumption per token for the Phi 3.5 Mini model across all tested configurations and devices. In most cases, the fastest configurations also exhibit the lowest energy consumption per token for both prefill and generation phases, as the reduction in inference time outweighs the increased power draw. However, some notable exceptions were observed. On the RPi 5, the *powersave* governor with optimal thread count resulted in lower energy consumption per token than the *ondemand* and *performance* governors due to its reduced power draw, despite the longer inference time. On the Orin GPU, the *15W* mode consumed up to 7% less energy per token than the *MAX* mode during generation phase, as the power savings compensated for the higher inference time. In contrast, the *MAX* mode resulted in in up to 25% energy consumption per token for all models up to Yi 1.5 6B during the prefill phase compared to the *15W* mode with the difference being generally more pronounced for smaller models. On the RPi 5, the optimal configuration in the prefill phase was 4 threads across all models, governors, and quantization schemes. However, in the generation phase, the




(a) RPi 5, *Q4_K_M* (b) Orin CPU, *Q4_K_M* (c) Orin GPU, *Q4_K_M*

(d) RPi 5, *Q4_0* (e) Orin CPU, *Q4_0* (f) Orin GPU, *Q4_0*

Fig. 9. Energy consumption per token for all configurations, quantization schemes, and devices

optimal configuration varied depending on the model and quantization scheme. Flan T5 Large was a clear outlier, where 4 threads minimized energy consumption for all governors and quantization schemes. For all other models, under *Q4_K_M* quantization, 2 threads provided the best energy efficiency for both the *performance* and *ondemand* governors, while the *powersave* governor performed best with 3 threads, with the only exception being Qwen 2 0.5B, for which 4 threads with the *powersave* governor and 3 threads with the other two governors minimized energy consumption. In the case of *Q4_0* quantization, 2 threads were optimal for all models under the *performance* and *ondemand* governors, while for the *powersave* governor either 3 or 2 threads were optimal depending on the model.

On the Orin CPU, the *powersave* governor generally exhibited higher energy consumption than the *performance* and *ondemand* governors, which showed no significant differences in energy usage. In the prefill phase, the maximum thread count of 6 proved to be optimal across all models, governors, and quantization schemes. For the generation phase, under *Q4_K_M* quantization, 5 threads yielded the best energy efficiency across all models and governors, except for Flan T5 Large with the *performance* and *powersave* governors, where 6 threads resulted in minimal energy consumption per token for the *ondemand* and *performance* governors. When considering the *Q4_0* quantization, 4 threads were optimal for the *performance* and *ondemand* governors, while 5 threads were best for the *powersave* governor across all models. The only exception was Flan T5 Large, which also showed optimal energy consumption with 5 threads under the *performance* and *ondemand* governors.

A comparison of quantization schemes reveals that *Q4_0* quantization significantly reduces energy consumption. In the prefill phase, energy usage is lowered by factors of $\sim 2.5$ to $\sim 5$ for CPU inference. In the generation phase, CPU energy consumption decreases by $\sim 10$–$\sim 70\%$, depending on the device and configuration. For GPU inference, energy consumption is reduced by $\sim 10$–$\sim 70\%$ during prefill phase and